

Paradigmas de Programação — Aula 08:

Concorrência, gorrotinas e canais

Canais

Canal

- **Canal** (*channel*): conduto tipado por onde uma gorrotina envia valores e outra os recebe

```
ch := make(chan string)
go func() {
    ch <- "ping"    // envia: bloqueia até alguém receber
}()
msg := <-ch        // recebe: bloqueia até alguém enviar
```

- **Sem *buffer***: envio e recebimento se encontram — o canal **comunica e sincroniza**
- **Com *buffer*** (`make(chan T, n)`): aceita até `n` envios sem receptor
- Todas as gorrotinas bloqueadas: `fatal error: all goroutines are asleep - deadlock!`

Produtor e consumidor: `close` e `range`

```
func produzir(n int, saida chan<- int) {    // chan<- : só envia
    for i := 1; i <= n; i++ {
        saida <- i
    }
    close(saida)                            // não vem mais nada
}

numeros := make(chan int)
go produzir(10, numeros)
soma := 0
for n := range numeros {                   // encerra no close
    soma += n
}
// soma == 55
```

- Nenhuma variável é acessada pelas duas gorrotinas: o dado **passa** pelo canal
- Só quem envia fecha o canal

select : o primeiro canal pronto

```
select {  
case r := <-respostas:  
    fmt.Println("chegou:", r)  
case <-time.After(100 * time.Millisecond):  
    fmt.Println("prazo esgotado")  
}
```

- Espera em vários canais e segue pelo primeiro que ficar pronto
- `time.After(d)` : canal que recebe um valor depois de `d` — implementa **prazo**
- `07-canaais.go` : resposta de 50 ms chega; resposta de 300 ms, `prazo esgotado`

O princípio do modelo

"Do not communicate by sharing memory; instead, share memory by communicating." — Effective Go

Threads (nota 02)	Gorrotinas e canais
várias threads acessam o mesmo dado	cada dado tem um único dono
o acesso é protegido por trava	as demais gorrotinas falam com o dono por canal
seção crítica a proteger	nada compartilhado, nada a travar